

#06 Sorting & Greedy Algorithm

김성수

4thrunyul@gmail.com

0. 목차



1. Sorting

- `std::sort`
- Sorting Various Data Types
- Custom Comparator
- Strict Weak Ordering
- `std::stable_sort`
- Counting Sort
- Radix Sort

2. Greedy Algorithm

- Conditions
- Proof

1. Sorting

1. Sorting

Sorting Algorithm(정렬 알고리즘)

- 데이터를 특정 기준(오름차순, 혹은 내림차순)에 따라 순서대로 재배열하는 작업
- 선택, 삽입, 버블 같은 $O(n^2)$, 퀵, 병합 같은 $O(n \log n)$, 계수, 기수 같은 $O(n)$ 알고리즘이 있음.

- 정렬 전

5	1	2	4	3
---	---	---	---	---

- 오름차순 정렬

1	2	3	4	5
---	---	---	---	---

- 내림차순 정렬

5	4	3	2	1
---	---	---	---	---

1. Sorting



PS에서의 정렬?

- 정렬 알고리즘을 직접 구현하라는 문제는 거의 없음
- 대부분 언어에서 제공하는 내부 함수를 통해 정렬 수행
- 정렬의 종류에 대해 알고 싶다면...
- @TimoBingmann [15 Sorting Algorithms in 6 Minutes](#)

1.1 std::sort

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {5, 2, 4, 3, 1};
9      int arr[] = {4, 3, 1, 2, 5};
10
11      sort(v.begin(), v.end());
12      sort(arr, arr + 5);
13
14      for(int i = 0; i < v.size(); i++) {
15          cout << v[i] << " ";
16      }
17
18      cout << "\n";
19
20      for(int i = 0; i < 5; i++) {
21          cout << arr[i] << " ";
22      }
23
24      return 0;
25 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- 지정한 범위에 속한 원소들을 오름차순으로 정렬
- 벡터의 경우 반복자(iterator)를, 배열의 경우 포인터를 인자로 넣어줘야 함

v.begin()

- 시작점, 벡터의 첫 번째 원소를 가리키는 반복자 반환

v.end()

- 끝점, 벡터의 마지막 원소 다음 칸을 가리키는 반복자 반환
- 만약 벡터의 크기가 n이라면 v.begin() + n는 v.end()와 같음

```
Input:                                     Copy
Expected Output:                           Copy
1 2 3 4 5
1 2 3 4 5
```

1.1 std::sort

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      vector<int> v = {5, 2, 4, 3, 1};
10
11      sort(v.begin(), v.end(), greater<>());
12
13      for(int i = 0; i < v.size(); i++) {
14          cout << v[i] << " ";
15      }
16
17      cout << "\n";
18
19      string s = "Piano";
20
21      sort(s.begin(), s.end());
22
23      cout << s << "\n";
24
25      reverse(s.begin(), s.end());
26
27      cout << s;
28
29      return 0;
30 }
```

sort(시작 위치, 끝 위치, greater<>()) | <algorithm> | $O(n \log n)$

- 지정한 범위에 속한 원소들을 내림차순으로 정렬

reverse(시작 위치, 끝 위치) | <algorithm> | $O(n)$

- 지정한 범위에 속한 원소들의 순서를 거꾸로 뒤집음

```
Input: Copy
Expected Output: Copy
5 4 3 2 1
Paino
oniaP
```

1.1 std::sort



실습(BOJ)

- [2750 수 정렬하기](#)
 - [코드](#)
- [11931 수 정렬하기 4](#)
 - [코드](#)

1.1 std::sort



실습(정올)

- [1346 대표값2](#)
- [8551 Tutorial : STL Sort 1](#)
- [8556 Tutorial : STL Reverse](#)

1.2 Sorting Various Data Types

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      int n;
9
10     cin >> n;
11
12     vector<int> v(n);
13
14     for(int i = 0; i < n; ++i) {
15         cin >> v[i];
16     }
17
18     sort(v.begin(), v.end());
19
20     for(int i : v) {
21         cout << i << " ";
22     }
23
24     return 0;
25 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- `vector<int>`에 대한 정렬 : 크기 순 정렬

입력:

복사

9
10 4 55 691 62 12 1 0 -5

예상 출력:

복사

-5 0 1 4 10 12 55 62 691

1.2 Sorting Various Data Types

```

1  #include <iostream>
2  #include <string>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      string s;
9
10     cin >> s;
11
12     sort(s.begin(), s.end());
13
14     cout << s;
15
16     return 0;
17 }

```

입력: 복사

Hanshin-Univ.

예상 출력: 복사

-.HUahinnnsv

sort(시작 위치, 끝 위치) | <algorithm> | $O(n \log n)$

- string, vector<char>에 대한 정렬 : ASCII Code순 정렬

10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자
0	0x00	NUL	32	0x20	Space	64	0x40	@	96	0x60	`
1	0x01	SOH	33	0x21	!	65	0x41	A	97	0x61	a
2	0x02	STX	34	0x22	"	66	0x42	B	98	0x62	b
3	0x03	ETX	35	0x23	#	67	0x43	C	99	0x63	c
4	0x04	EOT	36	0x24	\$	68	0x44	D	100	0x64	d
5	0x05	ENQ	37	0x25	%	69	0x45	E	101	0x65	e
6	0x06	ACK	38	0x26	&	70	0x46	F	102	0x66	f
7	0x07	BEL	39	0x27	'	71	0x47	G	103	0x67	g
8	0x08	BS	40	0x28	(	72	0x48	H	104	0x68	h
9	0x09	TAB	41	0x29	)	73	0x49	I	105	0x69	i
10	0x0A	LF	42	0x2A	*	74	0x4A	J	106	0x6A	j
11	0x0B	VT	43	0x2B	+	75	0x4B	K	107	0x6B	k
12	0x0C	FF	44	0x2C	,	76	0x4C	L	108	0x6C	l
13	0x0D	CR	45	0x2D	-	77	0x4D	M	109	0x6D	m
14	0x0E	SO	46	0x2E	.	78	0x4E	N	110	0x6E	n
15	0x0F	SI	47	0x2F	/	79	0x4F	O	111	0x6F	o
16	0x10	DLE	48	0x30	0	80	0x50	P	112	0x70	p
17	0x11	DC1	49	0x31	1	81	0x51	Q	113	0x71	q
18	0x12	DC2	50	0x32	2	82	0x52	R	114	0x72	r
19	0x13	DC3	51	0x33	3	83	0x53	S	115	0x73	s
20	0x14	DC4	52	0x34	4	84	0x54	T	116	0x74	t
21	0x15	NAK	53	0x35	5	85	0x55	U	117	0x75	u
22	0x16	SYN	54	0x36	6	86	0x56	V	118	0x76	v
23	0x17	ETB	55	0x37	7	87	0x57	W	119	0x77	w
24	0x18	CAN	56	0x38	8	88	0x58	X	120	0x78	x
25	0x19	EM	57	0x39	9	89	0x59	Y	121	0x79	y
26	0x1A	SUB	58	0x3A	:	90	0x5A	Z	122	0x7A	z
27	0x1B	ESC	59	0x3B	;	91	0x5B	[	123	0x7B	{
28	0x1C	FS	60	0x3C	<	92	0x5C	\	124	0x7C	
29	0x1D	GS	61	0x3D	=	93	0x5D	]	125	0x7D	}
30	0x1E	RS	62	0x3E	>	94	0x5E	^	126	0x7E	~
31	0x1F	US	63	0x3F	?	95	0x5F	_	127	0x7F	DEL

1.2 Sorting Various Data Types

```
1  #include <iostream>
2  #include <algorithm>
3  #include <vector>
4  #include <string>
5
6  using namespace std;
7
8  int main() {
9      vector<string> v = {
10         "banana", "apple",
11         "pear", "watermelon"
12     };
13     string s = "GoodProgram!";
14
15     sort(v.begin(), v.end());
16     sort(s.begin(), s.end());
17
18     for(int i = 0; i < v.size(); i++) {
19         cout << v[i] << "\n";
20     }
21
22     cout << s;
23
24     return 0;
25 }
```

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- vector<string>에 대한 정렬 : 사전순 정렬
- string에 대한 정렬 : ASCII Code순 정렬

Input: [Copy](#)

Expected Output: [Copy](#)

```
apple
banana
pear
watermelon
!GPAdgmooorr
```

1.2 Sorting Various Data Types

sort(시작 위치, 끝 위치) | `<algorithm>` | $O(n \log n)$

- `vector<pair<type1, type2>>`에 대한 정렬 : type1에 대한 정렬 이후, type2에 대한 정렬 수행

```
1  #include <iostream>
2  #include <vector>
3  #include <utility>
4  #include <string>
5  #include <algorithm>
6
7  using namespace std;
8
9  int main() {
10     vector<pair<string, int>> v = {
11         {"banana", 2},
12         {"apple", 1},
13         {"apple", 2},
14         {"pear", 1},
15         {"banana", 5}
16     };
17
18     sort(v.begin(), v.end());
19
20     for(int i = 0; i < v.size(); i++) {
21         cout << v[i].first << " ";
22         cout << v[i].second << "\n";
23     }
24
25     return 0;
26 }
```

Input: Copy

Expected Output: Copy

```
apple 1
apple 2
banana 2
banana 5
pear 1
```

1.2 Sorting Various Data Tyes



실습(BOJ)

- [11651 좌표 정렬하기 2](#)
 - [코드](#)

1.2 Sorting Various Data Tyes



실습(정올)

- [4885 숫자문자열 정렬](#)
- [8557 Tutorial : STL Pair](#)

1.3 Custom Comparator

Custom Comparator(사용자 정의 비교 함수)

- STL의 정렬 함수나 정렬 기반 컨테이너(set, map, priority_queue)에 적용할 수 있는 사용자 정의 비교 함수
- 구현 방식
 - 1. 일반 함수
 - 2. 구조체 내 operator<> 오버로딩
 - 3. 람다식
 - 4. 함수 객체
- 금일 자료에서는 1. 일반 함수 내용만 포함 됨

1.3 Custom Comparator

Custom Comparator(사용자 정의 비교 함수)

- STL의 정렬 함수나 정렬 기반 컨테이너(set, map, priority_queue)에 적용할 수 있는 사용자 정의 비교 함수
- 비교함수 $f(x, y)$ 는 정렬 과정에서 두 원소 x 와 y 의 위치 관계를 결정함
- 기본적인 형태 : `bool compare(const type& x, const type& y)`
- 입력 : 정렬 대상 컨테이너의 두 원소 x, y
- 출력 : [true, false]

- 반환값의 의미
 - $f(x, y) = true$
 - 원소 x 가 y 보다 반드시 앞(왼쪽)에 위치해야 함
 - $f(x, y) = false$
 - 원소 x 가 원소 y 보다 앞에 올 필요가 없음
 - 즉, y 가 x 보다 앞에 위치하거나 두 원소의 우선순위가 동등하여 순서를 바꿀 필요가 없음

- 다음 챕터에 나오는 Stric Weak Ordering을 위반하지 않게 작성해야 함.

1.3 Custom Comparator

```

1  #include <iostream>
2  #include <vector>
3  #include <utility>
4  #include <string>
5  #include <algorithm>
6
7  using namespace std;
8
9  bool compare(const pair<string, int>& a, const pair<string, int>& b) {
10     // 이름 오름차순 정렬(사전순)
11     if(a.first != b.first) {
12         return a.first < b.first;
13     }
14
15     // 개수 내림차순 정렬
16     return a.second > b.second;
17 }
18
19 int main() {
20     vector<pair<string, int>> v = {
21         {"banana", 2},
22         {"apple", 1},
23         {"apple", 2},
24         {"pear", 1},
25         {"banana", 5}
26     };
27
28     sort(v.begin(), v.end(), compare);
29
30     for(int i = 0; i < v.size(); i++) {
31         cout << v[i].first << " ";
32         cout << v[i].second << "\n";
33     }
34 }

```

sort(시작 위치, 끝 위치, 정렬 기준) | `<algorithm>` | $O(n \log n)$

- 만약 특정 원소는 오름차순으로, 다른 원소는 내림차순으로 정렬하고 싶다면?
- pair.first가 아니라 pair.second를 우선 정렬 기준으로 삼고 싶다면?
- 사용자 정의 비교 함수가 필요

사용자 정의 비교 함수

- bool compare(parameter a, parameter b)
- const : 원본 훼손 방지
- & : 인자를 복사해오는 게 아니라 주소를 지정

```

Input:                                     Copy
apple 2
apple 1
banana 5
banana 2
pear 1

Expected Output:                           Copy
apple 2
apple 1
banana 5
banana 2
pear 1

```

1.3 Custom Comparator



```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5
6  using namespace std;
7
8  struct Student {
9      string name;
10     int korean;
11     int math;
12 };
13
14 bool compare(const Student& a, const Student& b) {
15     // 이름 오름차순(사전순)
16     if(a.name != b.name) {
17         return a.name < b.name;
18     }
19
20     // 국어 점수 내림차순(높은 점수 순)
21     if(a.korean != b.korean) {
22         return a.korean > b.korean;
23     }
24
25     // 수학 점수 오름차순(점수 낮은 순)
26     return a.math < b.math;
27 }
28 }
```

sort(시작 위치, 끝 위치, 정렬 기준) | `<algorithm>` | $O(n \log n)$

- 정렬 요소가 3개 이상일 때는 구조체 사용
- pair와 다르게 정렬 연산자(operator)가 정의되어있지 않으므로, 비교 함수가 무조건 필요

1.3 Custom Comparator

```
30 int main() {
31     vector<Student> v = {
32         {"Bob", 90, 80},
33         {"Eric", 100, 70},
34         {"Alice", 90, 100},
35         {"Alice", 70, 80},
36         {"Eric", 100, 100},
37     };
38
39     sort(v.begin(), v.end(), compare);
40
41     for(auto it : v) {
42         cout << it.name << " ";
43         cout << it.korean << " ";
44         cout << it.math << "\n";
45     }
46
47     return 0;
48 }
```

sort(시작 위치, 끝 위치, 정렬 기준) | <algorithm> | $O(n \log n)$

- 정렬 요소가 3개 이상일 때는 구조체 사용
- pair와 다르게 정렬 연산자(operator)가 정의되어있지 않으므로, 비교 함수가 무조건 필요

Input: [Copy](#)

Expected Output: [Copy](#)

```
Alice 90 100
Alice 70 80
Bob 90 80
Eric 100 70
Eric 100 100
```

1.3 Custom Comparator



실습(정올)

- [8553 Tutorial : STL Sort 2 \(정렬 기준 설정\)](#)
- [8576 Tutorial : 구조체 연산자 오버로딩](#)
- [8563 Tutorial : STL Sort 3 \(구조체 정렬하기\)](#)
- [8577 회의 정렬](#)

1.4 Strick Weak Ordering

Strick Weak Ordering(엄격한 약한 순서 관계)

- STL의 정렬 함수나 정렬 기반 컨테이너(set, map, priority_queue)가 비교 함수에 요구하는 수학적 조건
- 해당 규칙을 지키지 않은 비교 함수를 std::sort에 적용하면 Segmentation Fault 등의 오류 발생
- 1. Irreflexivity(비반사성)
 - 자기 자신과의 비교는 항상 false여야 함. 즉, 자기 자신이 자기 자신보다 순서상 앞설 수 없음.
 - $f(x, x) = false$
- 2. Asymmerty(비대칭성)
 - x가 y보다 앞서면, y는 x보다 앞설 수 없음.
 - $f(x, y) = true \Rightarrow f(y, x) = false$

1.4 Strick Weak Ordering

Strick Weak Ordering(엄격한 약한 순서 관계)

- STL의 정렬 함수나 정렬 기반 컨테이너(set, map, priority_queue)가 비교 함수에 요구하는 수학적 조건
- 해당 규칙을 지키지 않은 비교 함수를 std::sort에 적용하면 Segmentation Fault 등의 오류 발생
- 3. Transitivity(이행성)
 - $x < y$ 이고, $y < z$ 이면, $x < z$ 이어야 함.
 - $f(x, y) = true, f(y, z) = true \Rightarrow f(x, z) = true$
- 4. Transitivity of Equivalence(동등성의 이행성)
 - x 와 y 가 같고, y 와 z 가 같다면, x 와 z 도 같아야 함.
 - 같다($x \equiv y$)의 정의는 $f(x, y) = false$ 이고 $f(y, x) = false$ 인 상태를 말 함.
 - $x \equiv y, y \equiv z \Rightarrow x \equiv z$

1.4 Strick Weak Ordering



```
bool compare(int a, int b) {  
    return a <= b;  
}
```

비반사성 위반

- $f(x, x) = true$ 반환
- 사용자 정의 비교함수에서는 =이 없어야 함.

1.4 Strick Weak Ordering



```
struct Point {  
    int x, y;  
};  
  
bool compare(const Point& a, const Point& b) {  
    if (a.x < b.x) return true;  
    if (a.y < b.y) return true;  
    return false;  
}  
  
bool compare(const Point& a, const Point& b) {  
    if (a.x != b.x) return a.x < b.x;  
    return a.y < b.y;  
}
```

비반사성 위반

- A(2, 5), B(3, 1)의 경우 문제 발생
- $A < B$ 와 $B > A$ 가 공존

1.5 std::stable_Sort

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <string>
5
6  using namespace std;
7
8  struct Student {
9      int score;
10     string name;
11 };
12
13 bool compare(const Student& a, const Student& b) {
14     return a.score < b.score;
15 }
16
17 int main() {
18     vector<Student> v = {
19         {5, "Bob"},
20         {3, "Eric"},
21         {5, "Alice"}
22     };
23
24     stable_sort(v.begin(), v.end(), compare);
25
26     for(int i = 0; i < v.size(); i++) {
27         cout << v[i].score << " " << v[i].name << "\n";
28     }
29 }
```

stable_sort(시작 위치, 끝 위치, 정렬 기준)

`<algorithm>` | $O(n \log n)$

- 특정 원소는 오름차순(혹은 내림차순)으로 정렬하되, 다른 원소는 입력 순서를 유지하고 싶다면
- ex) 왼쪽 코드는 점수순으로 오름차순 정렬하되, 점수가 같다면 입력 순서대로 출력하고 싶을 때
- std::sort는 퀵 정렬 기반인 것에 반해 std::stable_sort는 병합 정렬 기반

Input:	Copy
Expected Output:	Copy
3 Eric	
5 Bob	
5 Alice	

1.5 std::stable_Sort



실습(BOJ)

- [10814 나이순 정렬](#)
 - [코드](#)

1.6 Counting Sort



수 정렬하기 3

성공

언어 제한



시간 제한

메모리 제한

제출

정답

맞힌 사람

정답 비율

5 초 (하단 참고)

8 MB (하단 참고)

381652

93029

71240

24.188%

문제

N개의 수가 주어졌을 때, 이를 오름차순으로 정렬하는 프로그램을 작성하시오.

입력

첫째 줄에 수의 개수 N ($1 \leq N \leq 10,000,000$)이 주어진다. 둘째 줄부터 N개의 줄에는 수가 주어진다. 이 수는 10,000보다 작거나 같은 자연수이다.

출력

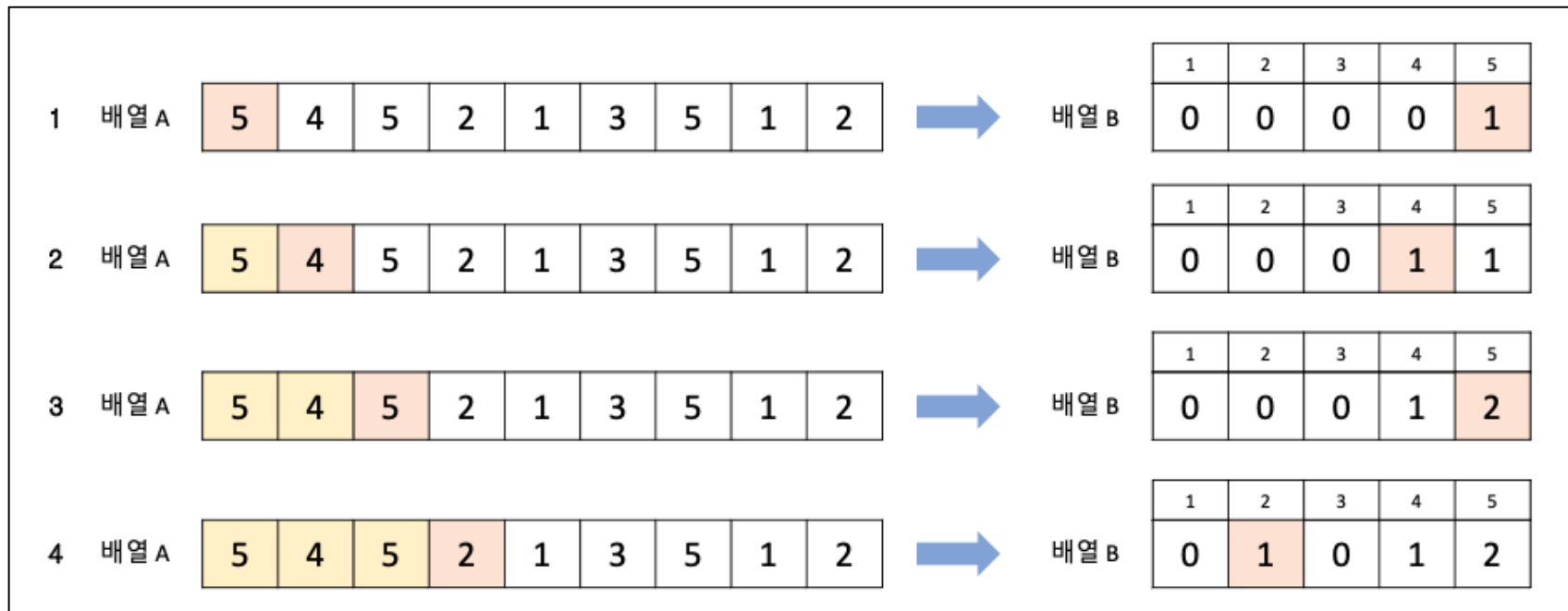
첫째 줄부터 N개의 줄에 오름차순으로 정렬한 결과를 한 줄에 하나씩 출력한다.

기존 문제와 비교했을 때 특이한 점?

1.6 Counting Sort

Counting Sort(계수 정렬) | $O(n + k)$

- 원소 간의 크기를 비교하지 않고, 각 원소가 몇 번 등장했는지 개수를 세어 정렬하는 알고리즘
- $O(n \log n)$ 의 이론적 한계를 넘어 데이터 범위가 작을 때 $O(n + k)$ 보장
 - 단, n : 원소의 개수, k : 데이터의 최대값 범위
- 데이터의 최대값(k)가 매우 커지면 메모리 초과(MLE) 발생
- 배열의 인덱스를 사용하므로, 실수 데이터에는 적용 불가



1.6 Counting Sort



실습(BOJ)

- [10989 수 정렬하기 3](#)
 - [코드](#)

1.6 Counting Sort



실습(정올)

- [3339 계수정렬\(counting sort\)1](#)

1.7 Radix Sort

Radix Sort | $O(d * (n + k))$

- 원소 간의 직접적인 크기 비교 없이, 자릿수(digit)별로 나누어 정렬하는 알고리즘
- 선형 시간 $O(d * (n + k))$ 보장.
 - 단, n : 원소 개수, d : 최대 자릿수, k : 기수(10진수의 경우 $k = 10$)
- 가장 작은 자릿수(1의 자리)부터 시작하여 기수 버킷에 넣어 정렬
- 정렬된 순서를 유지한 채 다음 자릿수(10의 자리, 100의 자리...)로 이동하여 정렬 반복
- 비트 연산자를 통해 최적화할 수 있음



2. Greedy Algorithm

2. Greedy Algorithm

Greedy Algorithm(욕심쟁이 알고리즘)

- 매 순간마다 지금 당장 가장 좋은 선택(Local Optima, 지역 최적해)을 이어나가 최종적인 해답(Global Optima, 전역 최적해)에 도달하는 알고리즘
- 단방향성 : 한 번 내린 결정을 번복하지 않음
- 효율성 : 모든 경우의 수를 따지지 않기 때문에 보통 $O(n)$ 이나 $O(n \log n)$ 으로 해결 가능
- 직관의 함정 : ‘왠지 이렇게 하면 해결 할 수 있지 않을까?’ 같은 직관으로 접근하기는 쉽지만, 증명하지 못하면 쉽게 틀림
 - PS 판의 유명한 격언
 - ‘아 이거 그리디 같은데’ $\rightarrow$ DP
 - ‘이건 진짜 DP다’ $\rightarrow$ 그리디

2.1 Greedy: Conditions



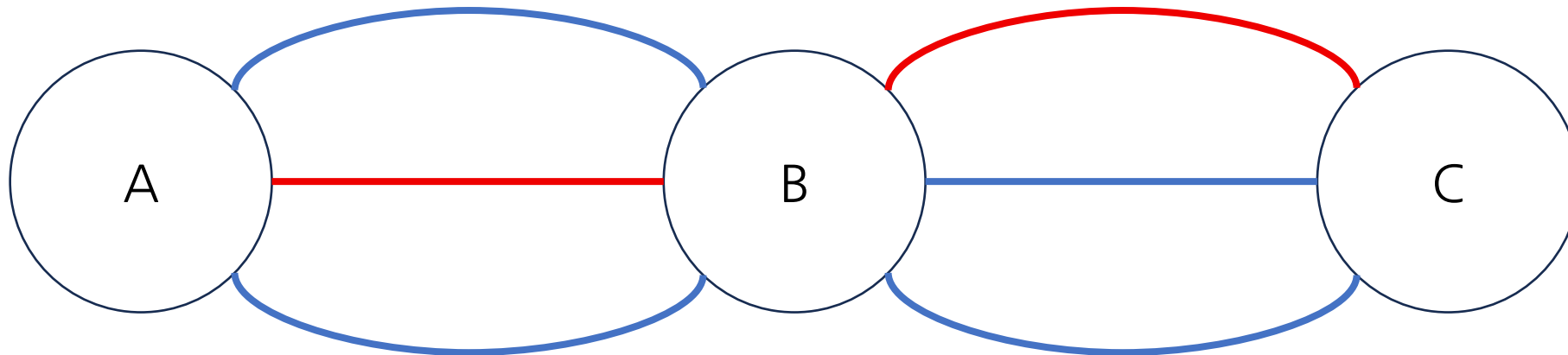
Conditions(성립 조건)

- 1. Greedy Choice Property(탐욕적 선택 속성)
 - 현재 단계에서 내린 탐욕적인 선택이 이후 단계의 선택에 영향을 주지 않는 성질
- 2. Optimal Substructure(최적 부분 구조)
 - 부분 문제의 최적 결과가 전체 문제의 최적 결과로 이어지는 구조

2.1 Greedy: Conditions

Shortest Path

- 해당 그래프에서 A-C의 최단 경로가 빨간 선 이라고 가정
- 최단 경로가 B를 통과한다고 가정
- A-B의 최단 경로와 B-C의 최단 경로 또한 빨간 선에 존재



2.1 Greedy: Conditions



실습(정올)

- [3521 Tutorial : 그리디\(Greedy - 탐욕, 욕심쟁이\) 알고리즘](#)
- [4520 햄버거 분배](#)
- [1370 회의실 배정](#)

2.2 Greedy: Proof



Proof(증명)

- 1. Proof by Contradiction(귀류법)
 - 그리디 선택이 최적해가 아니라고 가정한 뒤, 논리적 모순이 발생함을 보여 그리디 선택이 옳을 수 밖에 없음을 증명하는 방식
- 2. Mathematical Induction(귀납법)
 - 부분 문제의 최적 결과가 전체 문제의 최적 결과로 이어지는 구조
- 3. Exchange Argument(순서 교환 증명법)
 - 임의의 최적해에서 그리디 기준에 맞지 않는 두 요소를 교환해도 결과가 나빠지지 않음을 보여 그리디 해가 최적임을 증명하는 방식

2.2 Greedy: Proof



실습(정올)

- [8581 Tutorial : 그리디 증명 방법](#)
- [6271 구두 수선공](#)
- [6276 소방서의 고민](#)